GpsGate Data Pipeline - Project Handover

Project Overview

The GpsGate Data Pipeline is a Flask-based ETL system that extracts fleet tracking data from the GpsGate API, processes it, and stores it in a PostgreSQL database for business intelligence purposes. The system handles 8 different event types and maintains both fact and dimension tables.

Production URL: https://powerbixgpsgatexgdriver.onrender.com

System Architecture

Core Components

Web Application (`application.py`)

- Flask app factory pattern
- Multi-blueprint architecture
- Database connection management
- Environment configuration handling

Data Pipeline (`data_pipeline.py`)

- Event data processing engine
- API integration with GpsGate
- CSV parsing and data transformation
- Database storage with duplicate handling

Dashboard (`dashboard_enhanced.py`)

- Web-based monitoring interface
- Manual job triggers
- Real-time statistics
- Job execution tracking

Event Types Processed

1. **Trip** - Vehicle trip records
2. **Speeding** - Speed violation events
3. **Idle** - Vehicle idle time tracking
4. **AWH** - After work hours usage
5. **WH** - Work hours tracking
6. **HA** - Harsh acceleration events
7. **HB** - Harsh braking events
8. **WU** - Weekend usage patterns

Database Schema

Fact Tables

- `fact_trip` - Trip event records
- `fact_speeding` - Speeding violations
- `fact_idle` - Idle time events
- `fact_awh` - After hours usage
- `fact_wh` - Work hours tracking
- `fact_ha` - Harsh acceleration
- `fact_hb` - Harsh braking
- `fact_wu` - Weekend usage

Dimension Tables

- `dim_drivers` - Driver master data
- `dim_vehicles` - Vehicle information
- `dim_tags` - Tag/group definitions
- `dim_reports` - Available reports
- `dim_event_rules` - Event rule configurations
- `dim_vehicle_custom_fields` - Custom vehicle attributes

System Tables

- `job_executions` - Tracks all job runs
- `render` - API request cache
- `result` - API response cache

Configuration

Environment Variables

```
TOKEN=v2:MDAwMDAyODkwMDozOWM1MzQ1YWU2N2I4YWQ3MThhZA==
BASE_URL=https://omantracking2.com
DATABASE_URL=postgresql://[credentials]@[host]/[database]
SECRET_KEY=[flask-secret]
GDRIVE_FOLDER_ID=[google-drive-folder]
GOOGLE_APPLICATION_CREDENTIALS=[service-account-json]
```

API Configuration

All event types use standardized configuration that avoids server state issues:

- **Report ID**: 25 (for all events including Trip)
- **Tag ID**: 39
- **App ID**: 6

This configuration prevents the "reportId: 0" server errors that were encountered previously.

Operational Procedures

Scheduled Operations

Daily Sync (2:00 AM UTC)

- Syncs dimension tables from API
- Processes yesterday's fact data
- Handles 8 event types sequentially
- Records execution status

Weekly Backfill (Sunday 3:00 AM UTC)

- Processes last 7 days of data
- Reconciles any missing records
- Handles data quality issues
- Full fact table refresh

Manual Operations

Dashboard Access: `/dashboard/`

- Table record counts (fact and dimension)
- Manual sync triggers
- Job execution monitoring
- Scheduler status tracking

Manual Triggers:

- **Dimension Sync**: Updates all dimension tables from API
- **Fact Sync**: Processes fact data for specified date range
- **Full Backfill**: Complete dimension + fact sync

Data Flow

1. **Render Phase**: Submit request to GpsGate API
2. **Result Phase**: Retrieve and cache API response
3. **Download Phase**: Process CSV data
4. **Transform Phase**: Clean and validate records
5. **Load Phase**: Insert to database with duplicate handling

Deployment

Platform: Render.com

Services Deployed:

- Web service (Flask app)
- Daily sync cron job
- Weekly backfill cron job

Deployment Method: Git-based auto-deploy

- Repository: `Power-BI-X-GpsGate`
- Branch: `feature/completed-allMquery-Migration`

- Configuration: `render.yaml`

Build Process

```
pip install -r requirements.txt
```

Start Commands

```
# Web Service
gunicorn wsgi:app --workers 2 --threads 4 --timeout 600 --log-level info

# Daily Sync
cd /opt/render/project/src && python scheduled_daily_sync.py

# Weekly Backfill
cd /opt/render/project/src && python scheduled_weekly_backfill.py
```

File Structure

Production Files

- `wsgi.py` - WSGI entry point
- `application.py` - Flask app factory
- `config.py` - Environment configuration
- `models.py` - SQLAlchemy ORM models
- `data_pipeline.py` - Core ETL engine
- `dashboard_enhanced.py` - Web dashboard
- `render.py` - API client utilities
- `logger_config.py` - Logging configuration

Scheduled Jobs

- `scheduled_daily_sync.py` - Daily automation
- `scheduled_weekly_backfill.py` - Weekly reconciliation

Dimension Sync

- `sync_dimensions_from_api.py` - Dimension table updates

Manual Tools

- `backfill_2025_week1.py` - Historical data backfill
- `run_backfill_all_weeks.py` - Multi-week backfill orchestrator

Storage & Utilities

- `db_storage.py` - Database operations

- `fact_table_records.py` - Fact table utilities

Configuration

- `render.yaml` - Render deployment config
- `requirements.txt` - Python dependencies
- `Procfile` - Process definitions

API Endpoints

Data Processing

- `POST /wu-data` - WU event processing
- `POST /speeding-data` - Speeding event processing
- `POST /render` - Generic render endpoint
- `POST /result` - Result processing endpoint

Dashboard

- `GET /dashboard/` - Main dashboard interface
- `POST /dashboard/trigger/dimension-sync` - Manual dimension sync
- `POST /dashboard/trigger/fact-sync` - Manual fact sync
- `POST /dashboard/trigger/full-backfill` - Complete backfill
- `GET /dashboard/stats/table-counts` - Record count statistics
- `GET /dashboard/health/gpsgate-server` - Server health check

Troubleshooting

Common Issues

"reportId: 0" Error

- **Cause**: GpsGate server state issue
- **Solution**: Wait 10-15 minutes or use report_id=25 configuration
- **Prevention**: Current config avoids this issue

Missing Records

- **Check**: job_executions table for failed runs
- **Action**: Use manual backfill for specific date ranges
- **Tool**: Dashboard manual triggers

Cron Job Failures

- **Check**: Render service logs
- **Common**: Python path or import issues
- **Fix**: Verify startCommand includes full path

Monitoring

Health Checks:

- Dashboard shows table record counts
- Job execution history tracking
- Failed job error messages

Log Locations:

- Render service logs (web interface)
- Application logs (structured JSON format)
- Database job_executions records

Data Quality

Duplicate Handling: Automatic via database constraints **Data Validation**: Built into pipeline transformation **Error Recovery**: Failed records logged, processing continues

Business Context

Reporting Requirements

The system supports fleet management reporting by providing:

- Vehicle utilization metrics
- Driver behavior analysis
- Operational efficiency tracking
- Compliance monitoring

Data Freshness

- **Real-time**: Manual triggers available
- **Daily**: Yesterday's data by 2 AM
- **Weekly**: 7-day reconciliation every Sunday

Performance Characteristics

- **Concurrency**: 2 workers, 4 threads per worker
- **Timeout**: 10 minutes per request
- **Throughput**: ~1000 records/minute (varies by event type)
- **Storage**: PostgreSQL with connection pooling

Development Notes

Code Organization

- Blueprint-based modular architecture
- Environment-based configuration
- Comprehensive error handling and logging
- Database session management with proper cleanup

Recent Changes

- Unified all components to use reliable GpsGate API configuration

- Added dimension record tracking to dashboard
- Enhanced scheduled job reliability with better error handling
- Consolidated event processing logic across manual and scheduled operations

Extension Points

- Additional event types can be added via EVENT_CONFIG
- New dashboard widgets via additional API endpoints
- Custom reporting through database views
- Integration with external systems via webhook endpoints

Future Development Guide

Adding New Event Types

1. Update EVENT_CONFIG (3 files to modify):

```
# File: dashboard_enhanced.py (line 24)
# File: scheduled_daily_sync.py (line 44)
# File: scheduled_weekly_backfill.py (line 44)

EVENT_CONFIG = {
    'NewEventType': {
        'report_id': '25',           # Use 25 for reliability
        'event_id': 'XXX',          # Get from GpsGate API docs
        'response_key': 'new_event_data'
    }
}
```

2. Create Database Model (`models.py`):

```
class FactNewEvent(db.Model):
    __tablename__ = 'fact_new_event'
    # Add columns based on CSV structure
```

3. Update Dashboard Statistics (`dashboard_enhanced.py` line 465):

- Add new stat item in HTML template
- Update table count query

4. Add Migration (if using Alembic):

```
flask db migrate -m "Add fact_new_event table"
flask db upgrade
```

Adding New Dashboard Features

Location: `dashboard_enhanced.py`

New API Endpoint:

```
@dashboard_bp.route('/new-endpoint', methods=['GET'])
def new_function():
    # Implementation here
```

Frontend Integration (line 1000+ in HTML section):

```
async function newFeature() {
    const response = await fetch('/dashboard/new-endpoint');
    // Handle response
}
```

Database Schema Changes

Primary Location: `models.py`

- Add new models for additional tables
- Modify existing models (requires migration)

Migration Strategy:

1. Test changes locally first
2. Create database backup
3. Deploy during low-usage hours
4. Monitor for issues after deployment

Scheduled Job Modifications

Daily Sync Changes: `scheduled_daily_sync.py`

- Line 150+: Add new event types to processing list
- Line 130+: Modify dimension sync if needed

Weekly Backfill Changes: `scheduled_weekly_backfill.py`

- Line 140+: Add new event types
- Modify date range logic if required

Cron Schedule Changes: `render.yaml`

- Lines 19 & 29: Modify schedule expressions
- Use crontab.guru for schedule validation

API Configuration Updates

GpsGate API Changes:

- **Token Updates**: Environment variable `TOKEN`
- **New Endpoints**: Update `BASE_URL` or add new endpoints
- **Report ID Changes**: Modify EVENT_CONFIG across all files

Connection Issues:

- Check `render.py` timeout settings (line 50+)
- Modify retry logic in `data_pipeline.py`

Performance Optimization

Database Performance:

- Add indexes in migration files
- Monitor query performance via logs
- Consider partitioning for large tables

API Rate Limiting:

- Modify delays in `render.py`
- Adjust worker counts in `render.yaml`

Memory Usage:

- CSV processing chunking in `data_pipeline.py`
- Batch size adjustments for large datasets

Monitoring Enhancements

New Health Checks (`dashboard_enhanced.py`):

- Add endpoints similar to `/health/gpsgate-server`
- Database connection health
- External service availability

Alerting Integration:

- Webhook endpoints for external monitoring
- Email notifications for failed jobs
- Slack/Teams integration options

Security Considerations

Environment Variables:

- Rotate `TOKEN` regularly
- Update `SECRET_KEY` if compromised
- Review `DATABASE_URL` access

API Security:

- Add authentication to sensitive endpoints
- Implement rate limiting

- Review access logs regularly

Testing Strategy

Local Testing:

```
# Test imports
python -c "from scheduled_daily_sync import run_daily_sync; print('OK')"

# Test database connection
python -c "from application import db; print('DB OK')"

# Test API endpoints
curl https://powerbixgpsgatexgdriver.onrender.com/dashboard/
```

Staging Environment:

- Use separate Render service for testing
- Clone production database for testing
- Test scheduled jobs manually before deployment

Common Update Locations

Configuration Changes:

- Environment variables: Render dashboard
- API settings: EVENT_CONFIG in 3 files
- Database schema: models.py + migrations

Feature Additions:

- New endpoints: dashboard_enhanced.py
- New scheduled tasks: Copy existing cron job structure
- UI changes: HTML section in dashboard_enhanced.py

Bug Fixes:

- Logic issues: data_pipeline.py
- Display issues: dashboard_enhanced.py
- Scheduling issues: render.yaml + scheduled job files

Deployment Checklist

Before making changes:

1. ☑ Backup current database
2. ☑ Test changes locally
3. ☑ Update this documentation
4. ☑ Check syntax with Pylance
5. ☑ Commit with descriptive message

6. ☑ Monitor deployment logs
7. ☑ Verify functionality in production
8. ☑ Check scheduled jobs run successfully

Emergency Procedures

Rollback Steps:

1. Revert to previous Git commit
2. Force push to trigger re-deployment
3. Monitor for successful recovery

Database Recovery:

- Use Render database backup features
- Manual SQL restore if needed
- Re-run failed jobs from dashboard

Service Outage:

- Check Render service status
- Review application logs
- Verify environment variables
- Test database connectivity

FAQ & Onboarding Roadmap

Below are direct answers to common operational questions and a concise step-by-step onboarding roadmap for adding new customers or events.

1) Adding extra events for specific customers

- Where to change: `EVENT_CONFIG` in these files: `dashboard_enhanced.py`, `scheduled_daily_sync.py`, `scheduled_weekly_backfill.py`.
- Steps:
    1. Add the new event entry to `EVENT_CONFIG` (use `report_id=25` by default for reliability).
    2. Create a new `Fact` model in `models.py` following existing fact table patterns and add required columns.
    3. Add a migration (Alembic/Flask-Migrate) to create the table and any indexes.
    4. If CSV/response parsing differs, update parsing logic in `data_pipeline.py` to handle the new `response_key`.
    5. Update the dashboard UI (HTML in `dashboard_enhanced.py`) to display the new stat card and include the table in `get_table_counts()`.
    6. Test with a 1-day backfill and manually verify database rows and dashboard counts.

2) Passing dynamic Report ID / Tag ID / App ID per customer

- Recommended approach: store per-customer configuration in the database.

  - Create table `customer_configs`: `customer_id`, `app_id`, `token`, `base_url`, `tag_id`, `event_map` (JSON), `active`.
  - Scheduled jobs (`scheduled_daily_sync.py`, `scheduled_weekly_backfill.py`) iterate active customers and pass their config into `process_event_data`.
  - Dashboard/manual endpoints accept optional override fields (`app_id`, `report_id`, `tag_id`, `token`) that take precedence over DB config.

- Alternative for ad-hoc runs: pass `app_id`/`token`/`tag_id` directly in the API payload to the manual trigger endpoints.

3) Handling waiting time for report generation

- Flow: `render` request -> poll `result` endpoint -> when `ready` download CSV -> transform -> load.
- Best practices:
  - Implement polling with backoff (e.g., 5s, 15s, 30s) up to a sensible timeout (e.g., 10 minutes).
  - Cache `render` and `result` rows in `render`/`result` tables to avoid duplicate requests.
  - If API returns `reportId: 0` or similar server-state errors, mark job as degraded and retry later or use fallback report IDs.
  - Log render start/end timestamps and durations for observability.

4) Onboarding a new customer — step-by-step roadmap

Prerequisites: GpsGate credentials for the customer (token), example `app_id`, `report_id`, `tag_id`, access to the production DB for configuration.

Step-by-step:

1. Create customer configuration (DB row)
   - Insert into `customer_configs`:
     - `customer_id`: short id
     - `app_id`, `token`, `base_url`, `tag_id`
     - `event_map`: JSON mapping of event names -> `{report_id, event_id, response_key}`
     - `active`: true
2. Test API connectivity
   - Quick render test for a small date window using internal `/render` flow or Postman with customer credentials.
   - Confirm `result` becomes available and CSV is downloadable.
3. Initial dimension sync
   - Run dimension sync for the customer (it uses the shared dimension sync unless per-customer variations are required):

     ```
     python sync_dimensions_from_api.py
     ```

   - Verify `dim_` table counts and sample data.

4. Short fact sync (smoke test)
    - Run a 1-day fact sync using the dashboard manual trigger or internal backfill script with payload override for the customer.
    - Example: trigger `/dashboard/trigger/fact-sync` with body including `app_id`, `token`, `tag_id`.
5. Verify results
    - Check `GET /dashboard/stats/table-counts` and `GET /dashboard/status/recent`.
    - Spot-check data quality and timestamps.
6. Enable scheduled runs
    - If the customer should be included in the existing daily/weekly jobs, ensure their `active` flag is set.
    - For custom schedules, add a dedicated cron job in `render.yaml` or an external scheduler.
7. Document the onboarding entry (internal tracker)
    - Add credentials, test results, any anomalies, and the date of activation.

Quick commands and tests

Run daily sync locally (debug):

```
python scheduled_daily_sync.py
```

Run a one-day backfill (example):

```
python backfill_2025_week1.py --week_start 2025-01-01 --week_end 2025-01-02
```

Test a manual trigger (example curl):

```
curl -X POST https://powerbixgpsgatexgdriver.onrender.com/dashboard/trigger/fact-sync \\
  -H 'Content-Type: application/json' \\
  -d '{"start_date":"2025-01-01","end_date":"2025-01-02","app_id":"6","tag_id":"39","token":"v2:..."}'
```

Rollout and safety

- Use feature flag (`active` field) when onboarding multiple customers.
- Start with one customer in staging before enabling production.
- Monitor `job_executions` and `render` tables for unexpected errors.

Project Status: Active Production System **Last Updated**: February 2026 **Deployment**: Render.com (Auto-deploy enabled) **Monitoring**: Dashboard + Render service metrics **Next Review**: Recommended every 6 months